

MEMORY MATCH
FINAL SEMESTER EXAM PROJECT REPORT
(Prepared to fulfill the requirements of passing the Final Exam Semester 1)



By:
As'ad Maulana Yusuf (25031554171)
Rava Dwi Andika (25031554039)
Tabrizzi Aqlia (25031554008)

Lecturer:
Hasanuddin Al-Habib, S.Si., M.Si
Siska Puspitaningsih, S.kom., M.kom

STUDY PROGRAM DATA SCIENCE
FACULTY OF MATHEMATICS AND NATURAL SCIENCES
STATE UNIVERSITY OF SURABAYA
2025

TABLE OF CONTENTS

CHAPTER 1.....	3
INTRODUCTION.....	3
1.1 BACKGROUND.....	3
1.2 PROBLEM STATEMENT.....	3
1.3 OBJECTIVES.....	4
CHAPTER 2.....	6
ANALYSIS AND DESIGN.....	6
2.1 APPLICATION NECESSITY ANALYSIS.....	6
2.2 FLOWCHART.....	9
2.3 USER INTERFACE DESIGN SKETCH.....	11
CHAPTER 3.....	14
IMPLEMENTATION.....	14
3.1 CODE EXPLANATION.....	14
3.2 APPLICATION SCREENSHOTS.....	27
CHAPTER 4.....	33
ATTACHMENT.....	33
4.1 Embed File.....	33
REFERENCES.....	34

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

In the context of modern education and digital interaction, serious games have emerged as highly valuable tools for cognitive skill training. The Memory Match game is one of the well-established methods that fundamentally trains working memory, selective attention, and visual-spatial skills. Studies indicate that cognitive training through challenging games can enhance brain plasticity and executive functions.

However, digital implementations of memory games are often static, offering limited variation in difficulty levels or visual customization. These limitations reduce long-term appeal (replayability) and hinder the game's adaptability to users with diverse ability levels.

This project aims to address these gaps by developing a Memory Match Game using the Pygame library with the Python programming language. The project not only replicates the basic card-matching mechanism but also significantly expands its functionality:

1. **Difficulty Adaptability:** Providing three standard modes (Easy 4×4, Medium 4×6, Hard 6×6) tailored to different memory capacities.
2. **Visual Dynamics:** Implementing a dynamic system for loading themes and backgrounds.
3. **Performance Metrics:** Integrating a holistic scoring system that records completion time, number of moves, and awards stars based on efficiency.
4. **Special Challenges:** Introducing Nightmare Mode, a challenge mode designed to test players' concentration limits through potentially more complex board mechanics.

This development applies an Object-Oriented Programming (OOP) approach to ensure code modularity, ease of maintenance, and support for future feature expansion. Consequently, the game is expected to serve as an effective and engaging edutainment platform.

1.2 PROBLEM STATEMENT

Based on the background and development objectives, this project formulates several key technical problems that must be addressed:

1. **How to design the Pygame class architecture (Card, Board, Game)** to manage card states (open, closed, matched) and implement a smooth pseudo-3D flip animation while maintaining a stable frame rate.

2. **How to implement a dynamic asset-loading system** (`load_theme_images`) that is failure-tolerant (e.g., produces placeholders or a default card back when assets are missing) and can proportionally adjust card dimensions to fit all grid modes.
3. **How to build a State Machine within the Pygame game loop** to handle transitions between menus (Main Menu, Settings, Leaderboard) and a Pause Menu separately from the core gameplay logic.
4. **How to develop robust data persistency functions** (`save_score`, `load_scores`) to record performance metrics (time, moves, stars) into CSV files and present Leaderboard data with multi-priority sorting criteria.
5. **How to design and implement interactive UI components** (**Button**, **Slider**) that provide clear visual feedback (hover states) and functional control for audio volume through an `AudioController`.

1.3 OBJECTIVES

1.3.1 Project Objectives

1. Develop a 2D Pygame-based Memory Match Game software that runs stably at **60 FPS**.
2. Fully implement the core game logic, including the **check_match** mechanism and the **flip-back delay** (`FLIP_BACK_DELAY`) within the **Game** class.
3. Provide a **Leaderboard management system** capable of storing and displaying players' best scores for each difficulty mode.
4. Produce a clean and intuitive **UI design**, with functional music and SFX volume controls accessible through the **Settings** menu.

1.3.2 Project Constraints

1. The application is developed and optimized exclusively for **desktop environments** and operates at a fixed screen resolution of **800×600 pixels**.
2. Theme compatibility testing is limited to **PNG/JPG image formats** and **MP3/WAV audio assets**.
3. The **NightmareBoard** mechanism and its associated **NightmareCard** currently serve only as conceptual placeholders and do not yet implement

complex game-modifying logic (such as mid-game card repositioning), as the primary focus is on validating the core system.

4. Data storage uses only **CSV file I/O mechanisms**; integration with more advanced database systems (such as SQLite) is outside the scope of this project.

CHAPTER 2

ANALYSIS AND DESIGN

2.1 APPLICATION NECESSITY ANALYSIS

The Memory Match Game application is developed to support cognitive training in an interactive and structured way. The needs of the application can be analyzed from both the user side and the system side.

1. User Needs

Users need an application that can:

- Train visual memory and concentration in an enjoyable way : Users need a game that helps them practice remembering the position of cards. When they try to match card pairs, they use visual memory and focus. This improves their cognitive skills in a fun way.
- Offer different difficulty levels so players can choose a challenge suitable to their skills : Not all players have the same skill level. Some are beginners, others want a harder challenge. The game must provide *Easy*, *Medium*, and *Hard* modes so users can choose a difficulty that matches their ability. This also allows them to improve gradually.
- Provide visual themes that make the game more interesting and prevent boredom : Players may feel bored if the game always looks the same. Themes like *Animals*, *Flower*, *Emoji*, *Dress*, *Disney*, *Football* make the game more visually appealing and help maintain interest during long-term use.
- Use time-based scoring so users can see how fast they can complete the game : Users want to know how fast they can finish the game. A timer gives direct feedback about their speed, which is important for cognitive training. This scoring method is more meaningful than counting moves.
- Give star ratings as motivation to improve performance : A star system (★, ★★, ★★★) gives players a sense of achievement. They will try to finish faster next time to get more stars. This encourages continuous improvement.
- Save past performance (time, mode, theme, stars) so users can monitor progress : Users want to see whether they are improving over time. Saving data—such

as completion time, chosen mode, theme, and star rating—allows players to compare results and monitor their progress.

2. System Needs

The system must be able to:

- Display menus and buttons clearly : The system must show all menus (difficulty, theme, main menu) in a simple and readable layout so users can easily navigate the game.
- Generate card pairs based on the selected difficulty : Each difficulty mode contains a different number of cards (Easy 4×4, Medium 6×6, Hard 8×8). The system must create the correct number of card pairs for the selected mode.
- Run a timer accurately : The system needs a timer that starts when the game begins and stops when all card pairs are matched. Accurate timing is essential for the scoring system.
- Detect matches between cards : The system must check whether the two flipped cards have the same image. If they match: keep them open. If not: flip them back after a short moment.
- Calculate star ratings based on the time limit : Based on the user's completion time, the system must give 1–3 stars according to preset thresholds for each difficulty mode.
- Store performance history into a CSV file : The system must automatically save gameplay results (timestamp, mode, theme, time, stars) into a CSV file so the data can be viewed later.
- Provide a stable and responsive interface on all devices : The application must run smoothly without lag. Buttons should respond quickly, and the layout should work correctly on different device sizes.

3. Functional Requirements

The application must include:

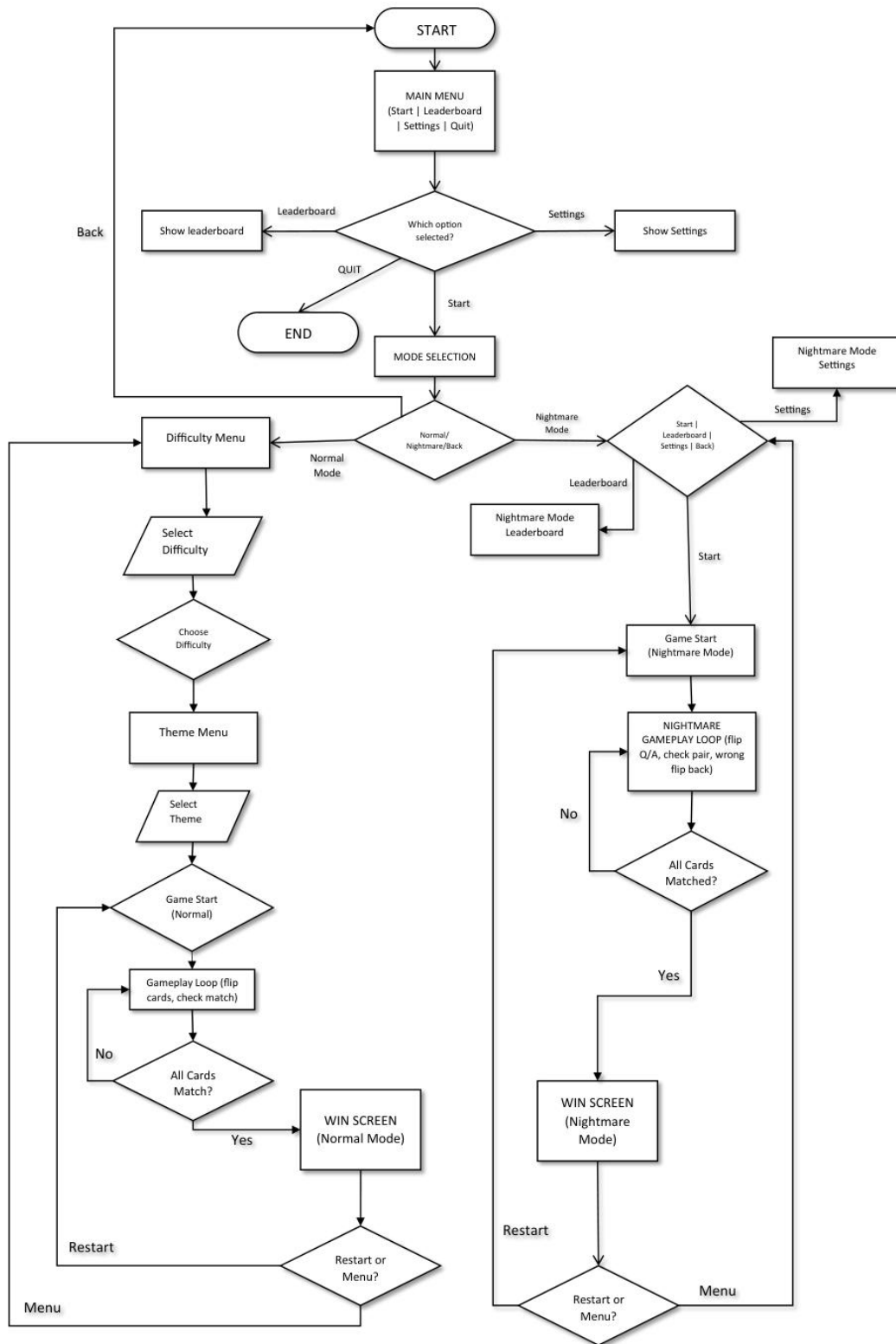
- Main Menu : The system must display the main screen where the user can start the game, choose difficulty, choose a theme, view records, or exit.
- Difficulty menu (Easy, Medium, Hard) : The system must show three difficulty choices: Easy, Medium, Hard, and apply the selected difficulty to the game.

- Theme selection menu : The system must allow users to choose a visual theme. The selected theme must update the appearance of the card images.
- Game screen with cards grid : The system must show the game board (grid of facedown cards) based on the selected difficulty and theme.
- Timer function : The system must start counting time when the game begins and stop when the last pair is matched.
- Match-checking logic : When two cards are flipped, the system must check whether they match, If match → keep them open, If not → flip them back.
- Result screen with stars : After finishing the game, the system must show, Total time, Number of stars, Difficulty mode, Theme
- CSV data saving function : After showing results, the system must save the performance data into a CSV file.

4. Non-Functional Requirements

- Easy to use : The interface must be simple and clear so users can play without needing instructions.
- Attractive visual design : The interface should be visually appealing. Colors, themes, and card images must be pleasant and well-organized.
- Fast response and smooth interaction : The system must not lag or freeze. All buttons and cards should respond immediately when clicked.
- Data stored safely : The saved CSV file must be protected from accidental deletion, and the system must write data correctly every time.
- Interface consistent for all themes : Even when the theme changes, the layout, button placement, and interaction style must remain consistent to avoid confusing the user.

2.2 FLOWCHART



The displayed flowchart illustrates the overall workflow of the Memory Match Game application, starting from the initial process until the game ends. This flow is designed to ensure that user interaction with the system is structured, logical, and repeatable according to the game requirements.

The process begins at the Start state, indicating that the application is launched by the user. The system then displays the Main Menu, which contains several primary options: starting the game (Start), viewing the leaderboard (Leaderboard), accessing the settings (Settings), and exiting the application (Quit). The system performs a decision-making process based on the user's selection. If the user chooses the Leaderboard option, the system displays the player score or achievement information and then returns the user to the main menu. If the user selects Settings, the system shows the application settings page and subsequently returns to the main menu. If the user selects Quit, the system immediately terminates the process and the application ends. When the user selects Start, the system proceeds to the game mode selection stage.

At the Mode Selection stage, the user is given the option to choose between Normal Mode, Nightmare Mode, or return to the previous menu. Selecting Normal Mode leads the user to the standard gameplay flow, while selecting Nightmare Mode directs the user to a more advanced gameplay flow with additional mechanics.

In Normal Mode, the system first displays the difficulty selection menu. After the difficulty level is determined, the system proceeds to the theme selection menu, where the user chooses the visual theme for the game cards. Once all initial configurations are completed, the system starts the game and enters the main gameplay loop. During this stage, the player flips cards and the system checks for matching pairs. This process runs iteratively within a gameplay loop. The system continuously evaluates whether all cards have been successfully matched. If not, the gameplay continues. Once all cards are matched, the system displays the Normal Mode Win Screen, after which the user is given the option to restart the game or return to the main menu.

In Nightmare Mode, the system provides a dedicated menu that includes options to start the game, view the Nightmare Mode leaderboard, access Nightmare Mode settings, or return to the previous menu. When the user chooses to start the game, the system initiates the Nightmare Mode gameplay. In this mode, the gameplay mechanics are more complex than in

Normal Mode. In addition to flipping and matching cards, players are presented with questions or challenges. The system checks both the card match and the correctness of the player's answers. If the cards do not match or the answer is incorrect, the cards are flipped back. This gameplay loop continues until the system detects that all cards have been successfully matched. Once this condition is met, the system displays the Nightmare Mode Win Screen. Similar to Normal Mode, the user can choose to restart the game or return to the main menu.

Overall, this flowchart demonstrates that the system is designed with a modular and iterative structure, allowing users to easily navigate between menus, configure gameplay settings, and replay the game without restarting the application. This structure supports a consistent user experience and facilitates future system development and maintenance.

2.3 USER INTERFACE DESIGN SKETCH

The User Interface Design Sketch shows the basic layout and structure of the screens used in the Memory Match Game application. These sketches help visualize how the user interacts with the system and how each part of the application is organized. The goal of the UI design is to make the game easy to understand, simple to use, and visually clear for all types of users. Below is the explanation for each UI screen:

1. Main Menu Screen

The Main Menu is the first screen the user sees when opening the application. Its purpose is to provide quick access to all main features of the game.

Key elements:

- **Start Game** – begins the gameplay using the previously selected difficulty and theme.
- **Difficulty** – takes the user to the difficulty selection screen.
- **Theme** – takes the user to the theme selection page to choose card visuals.
- **View Records** – opens the page that displays past performance saved in CSV.
- **Exit** – closes the application.

Purpose:

To guide the player to all main functions in a clear and organized manner.

2. Difficulty Selection Screen

This screen allows users to choose the challenge level before starting the game.

Options:

- **Easy** – small grid (4×4), fast and simple.
- **Medium** – medium grid, more pairs to match.
- **Hard** – large grid (8×8), requires more focus and time.

Back button:

Returns to the main menu if the user changes their mind.

Purpose:

To give players control over the difficulty and cognitive load they want to experience.

3. Theme Selection Screen

This page lets users choose the visual theme for the cards.

Purpose:

Themes are included to increase visual variety, reduce boredom, and improve user engagement.

Different themes also help personalize the game experience according to user preference.

4. Game Screen

This is the main interactive screen where players play the Memory Match Game.

Key elements:

- **Timer** – shows how long the player has been playing; important for scoring.
- **Difficulty & Theme Label** – indicates the mode and theme chosen.
- **Card Grid** – a set of face-down cards arranged according to difficulty level.
- **Pause and Quit Buttons** – allow the player to pause the game or exit.

Purpose:

To provide a clean and functional layout where players can focus on gameplay without distractions.

5. Pause Screen

This screen appears when the player pauses the game.

Options include:

- **Resume** – continue the game.
- **Restart** – begin the same game again from the start.
- **Main Menu** – return to the main menu without finishing the current game.

Purpose:

To give users control while playing, especially if they need to stop temporarily.

6. Results Screen

This screen appears after the player successfully matches all card pairs.

Displayed information:

- **Completion Time** – total time needed to finish the game.
- **Star Rating** – shows performance based on time (1–3 stars or none).
- **Difficulty & Theme** – summary of the chosen settings.

Buttons:

- **Play Again** – start a new game with the same settings.
- **Main Menu** – return to the home screen.

Purpose:

To provide feedback to the user and motivate improvement through star ratings.

7. Records / History Screen

This screen displays past game results recorded in a CSV file.

Data shown:

- Date and time
- Difficulty mode
- Theme
- Completion time in seconds
- Star rating

Purpose:

To help players track progress over time, making the game more meaningful and motivating. It also adds value as a simple data-logging feature.

CHAPTER 3

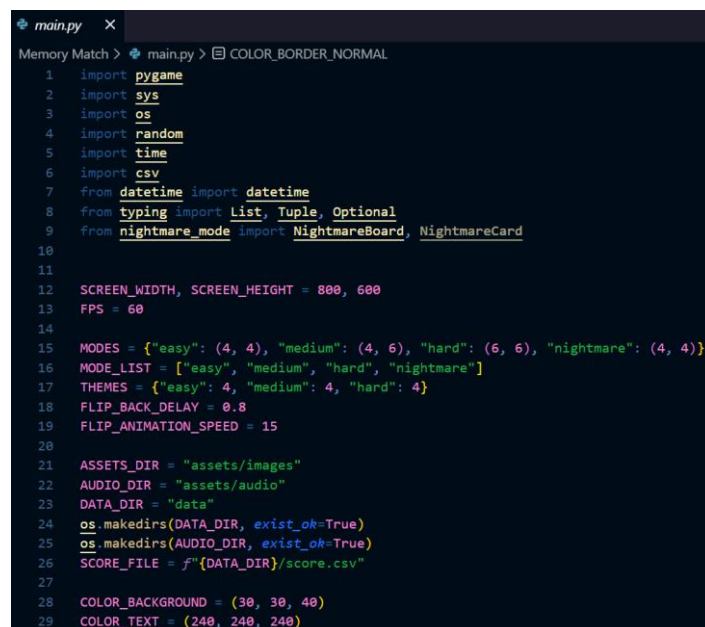
IMPLEMENTATION

3.1 CODE EXPLANATION

The implementation of this memory card game is primarily developed using the Pygame library in Python, focusing on modularity and the use of classes to represent game objects and manage game states. This code explanation is divided into key functional areas and main classes.

3.1.1 Initial Declarations and Configuration

The initial section of the code defines global constants and imports the necessary libraries, establishing the fundamental parameters for the game environment.



```
1 import pygame
2 import sys
3 import os
4 import random
5 import time
6 import csv
7 from datetime import datetime
8 from typing import List, Tuple, Optional
9 from nightmare_mode import NightmareBoard, NightmareCard
10
11
12 SCREEN_WIDTH, SCREEN_HEIGHT = 800, 600
13 FPS = 60
14
15 MODES = {"easy": (4, 4), "medium": (4, 6), "hard": (6, 6), "nightmare": (4, 4)}
16 MODE_LIST = ["easy", "medium", "hard", "nightmare"]
17 THEMES = {"easy": 4, "medium": 4, "hard": 4}
18 FLIP_BACK_DELAY = 0.8
19 FLIP_ANIMATION_SPEED = 15
20
21 ASSETS_DIR = "assets/images"
22 AUDIO_DIR = "assets/audio"
23 DATA_DIR = "data"
24 os.makedirs(DATA_DIR, exist_ok=True)
25 os.makedirs(AUDIO_DIR, exist_ok=True)
26 SCORE_FILE = f"{DATA_DIR}/score.csv"
27
28 COLOR_BACKGROUND = (30, 30, 40)
29 COLOR_TEXT = (240, 240, 240)
```

Explanation:

- **Module Imports:** Standard Python modules (os, random, time, csv, datetime) are imported along with the main game library, pygame. The inclusion of NightmareBoard and NightmareCard suggests an advanced game mode is defined separately to maintain code separation.
- **Game Constants:** Defines screen dimensions, the target frame rate (FPS), and the grid configurations for each game mode in MODES. The FLIP_BACK_DELAY constant is crucial for gameplay flow, controlling the pause after a failed match.
- **Directory Setup:** Path variables are set up for organizing assets and game data. The os.makedirs calls ensure that the necessary directories for persistent storage (DATA_DIR) and audio files exist before the game attempts to use them.

- **Colors and Rules:** Various RGB tuples define the color scheme for the UI. STAR_RULES is a core game mechanic, outlining the time benchmarks required to achieve 1, 2, or 3 stars for each difficulty level, incentivizing faster completion times.

3.1.2 Score Data Management Functions

These functions handle the initialization, saving, and loading of player scores, ensuring persistent record-keeping for the leaderboard feature using the CSV file format.

```
main.py X
Memory Match > main.py > COLOR_BORDER_NORMAL

56 def init_csv() -> None:
57     if not os.path.exists(SCORE_FILE):
58         try:
59             with open(SCORE_FILE, 'w', newline='') as f:
60                 writer = csv.writer(f)
61                 writer.writerow(['timestamp', 'mode', 'theme', 'time_seconds', 'moves', 'stars'])
62                 print("Score file initialized successfully")
63             except Exception as e:
64                 print(f"Error initializing score file: {e}")
65
66 def save_score(mode: str, theme: int, time_seconds: int, moves: int, stars: int) -> None:
67     try:
68         with open(SCORE_FILE, 'a', newline='') as f:
69             writer = csv.writer(f)
70             timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
71             writer.writerow([timestamp, mode, theme, time_seconds, moves, stars])
72             print(f"Score saved: {mode}, {theme} {time_seconds}s, {moves} moves, {stars} stars")
73     except Exception as e:
74         print(f"Error saving score: {e}")
```

```
score.csv X
Memory Match > data > score.csv > data

1 timestamp,mode,theme,time_seconds,moves,stars
2 2025-11-27 12:05:49,easy,3,51,16,1
3 2025-11-27 12:07:33,easy,4,27,17,2
4 2025-11-28 09:44:39,nightmare,1,73,35,2
5 2025-11-29 21:00:45,nightmare,1,40,21,3
```

Explanation:

- **init_csv(): Score File Initialization.** Ensures the score.csv file exists. If it doesn't, it creates the file and writes the header row defining the data schema.
- **save_score(): Score Persistence.** Appends a new record to the CSV file. It records the game outcome, including time taken, moves used, stars earned, and the current timestamp. Error handling is included to manage potential file I/O issues.
- **load_scores(): Data Retrieval.** Reads all records from the CSV file. It uses csv.DictReader to treat each row as a dictionary, making column access easier. Crucially, it converts time, moves, and stars from strings (CSV default) to integers for mathematical comparison later.
- **get_top_scores(): Leaderboard Generation.** Filters the loaded scores by an optional mode. It then sorts the scores using a lambda function that prioritizes higher stars first.

(-x['stars']), followed by the fastest time (x['time_seconds']), and then the fewest moves (x['moves']). This provides a comprehensive and fair ranking system.

3.1.3 Asset Loading Functions (Images and Audio)

This module is dedicated to fetching and preparing game assets from the file system for use by Pygame, handling image scaling, background loading, and sound setup.

```
main.py X
Memory Match > main.py > COLOR_BORDER_NORMAL
187 def load_theme_images(mode: str, theme: int) -> Tuple[list(Tuple), Optional[pygame Surface]]:
188     folder = f"{ASSETS_DIR}/{mode}/{theme}"
189     imgs = []
190     back = None
191
192     try:
193         if os.path.isdir(folder):
194             for f in os.listdir(folder):
195                 if f.lower().endswith(('.png', '.jpg', '.jpeg')):
196                     p = os.path.join(folder, f)
197                     try:
198                         surf = pygame.image.load(p).convert_alpha()
199                         if f.startswith("back"):
200                             back = surf
201                     except:
202                         imgs.append((surf, f))
203             except Exception as e:
204                 print(f"Error loading image {p}: {e}")
205
206     if back is None:
207         print(f"Warning: Back image not found in {folder}, creating default")
208         back = create_default_back_image()
209
210     pairs_needed = (MODES[mode][0] + MODES[mode][1]) // 2
211     while len(imgs) < pairs_needed:
212         print(f"Warning: Not enough images in {folder}, creating placeholder")
213         imgs.append(create_placeholder_image(len(imgs) + 1))
214
215     return imgs, back
```

```
main.py X
Memory Match > main.py > COLOR_BORDER_NORMAL
219
220 def load_background_image(path: str, default_color: Tuple = COLOR_BACKGROUND) -> Optional[pygame Surface]:
221     if os.path.exists(path):
222         try:
223             bg = pygame.image.load(path).convert()
224             bg = pygame.transform.scale(bg, (SCREEN_WIDTH, SCREEN_HEIGHT))
225             return bg
226         except Exception as e:
227             print(f"Error loading background {path}: {e}")
228     else:
229         print(f"Warning: Background not found at {path}")
230     return None
231
232 def load_menu_background() -> Optional[pygame Surface]:
233     return load_background_image(f"{ASSETS_DIR}/menu_background.jpg")
234
235 def load_difficulty_background() -> Optional[pygame Surface]:
236     return load_background_image(f"{ASSETS_DIR}/difficulty_background.jpg")
237
238 def load_leaderboard_background() -> Optional[pygame Surface]:
239     return load_background_image(f"{ASSETS_DIR}/leaderboard_background.jpg")
240
241 def load_settings_background() -> Optional[pygame Surface]:
242     return load_background_image(f"{ASSETS_DIR}/settings_background.jpg")
243
244 def load_theme_background() -> Optional[pygame Surface]:
245     return load_background_image(f"{ASSETS_DIR}/theme_background.jpg")
246
247 def load_game_background(mode: str, theme: int) -> Optional[pygame Surface]:
```

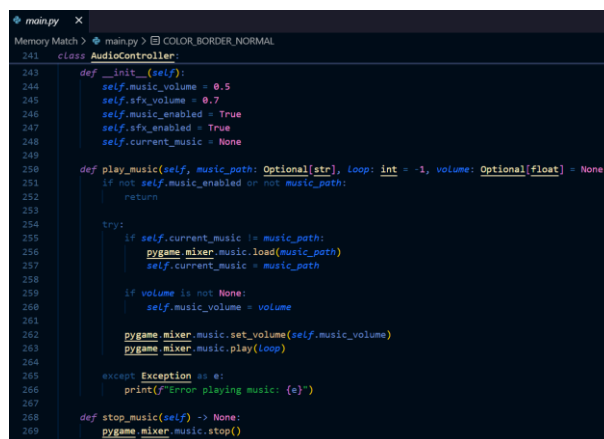
```
main.py X
Memory Match > main.py > COLOR_BORDER_NORMAL
197
198 def load_menu_music() -> Optional[str]:
199     music_path = f"{AUDIO_DIR}/menu_music.mp3"
200     if os.path.exists(music_path):
201         return music_path
202     print(f"Warning: Menu music not found at {music_path}")
203     return None
204
205 def load_game_music(mode: str, theme: int) -> Optional[str]:
206     music_path = f"{AUDIO_DIR}/{mode}_theme{theme}.mp3"
207     if os.path.exists(music_path):
208         return music_path
209
210     music_path = f"{AUDIO_DIR}/{mode}_music.mp3"
211     if os.path.exists(music_path):
212         return music_path
213
214     print(f"Warning: Game music not found for {mode} theme {theme}")
215     return None
216
217 def load_sound_effects() -> dict:
218     sounds = {}
219
220     sound_files = {
221         'flip': 'card_flip.wav',
222         'match': 'card_match.wav',
223         'win': 'win.wav',
224     }
```


Explanation:

- `load_theme_images(): Card Asset Preparation.`
 - Scans the specific theme directory (`assets/images/[mode]/theme[theme]`).
 - It separates the unique front-facing card images (`imgs`) from the single card-back image (`back`).
 - **Error Tolerance:** If the back image is missing, a simple default image is generated. If there are insufficient unique card images to fill the board (based on `MODES`), placeholder images are programmatically generated to prevent runtime errors, ensuring the game remains playable.
- `load_game_background(): Background Loading.` Provides a dedicated function to load the appropriate background image, handling scaling and special cases like the "nightmare" mode background.
- `load_sound_effects(): Sound Setup.` Loads predefined sound effect files (`.wav`) into `pygame.mixer.Sound` objects, which are managed by the `AudioController`.

3.1.4 AudioController Class

This class acts as a central control system for all music and sound effects, allowing for global volume and toggle settings.



```
main.py
Memory Match > main.py > COLOR_BORDER.NORMAL
241 class AudioController:
242     def __init__(self):
243         self.music_volume = 0.5
244         self.sfx_volume = 0.7
245         self.music_enabled = True
246         self.sfx_enabled = True
247         self.current_music = None
248
249     def play_music(self, music_path: Optional[str], loop: int = -1, volume: Optional[float] = None):
250         if not self.music_enabled or not music_path:
251             return
252
253         try:
254             if self.current_music != music_path:
255                 pygame.mixer.music.load(music_path)
256                 self.current_music = music_path
257
258             if volume is not None:
259                 self.music_volume = volume
260
261             pygame.mixer.music.set_volume(self.music_volume)
262             pygame.mixer.music.play(loop)
263
264         except Exception as e:
265             print(f"Error playing music: {e}")
266
267     def stop_music(self) -> None:
268         pygame.mixer.music.stop()
```

```

main.py x
Memory Match > main.py > COLOR_BORDER_NORMAL
241 class AudioController:
271     def play_sound(self, sound: Optional[pygame.mixer.Sound]) -> None:
272         if self.sfx_enabled and sound:
273             try:
274                 sound.set_volume(self.sfx_volume)
275                 sound.play()
276             except Exception as e:
277                 print(f"Error playing sound: {e}")
278
279     def set_music_volume(self, volume: float) -> None:
280         self.music_volume = max(0.0, min(1.0, volume))
281         pygame.mixer.music.set_volume(self.music_volume)
282
283     def set_sfx_volume(self, volume: float) -> None:
284         self.sfx_volume = max(0.0, min(1.0, volume))
285
286     def toggle_music(self) -> None:
287         self.music_enabled = not self.music_enabled
288         if not self.music_enabled:
289             self.stop_music()
290         else:
291             if self.current_music:
292                 try:
293                     pygame.mixer.music.load(self.current_music)
294                     pygame.mixer.music.set_volume(self.music_volume)
295                     pygame.mixer.music.play(-1)
296                 except Exception as e:
297                     print(f"Error replaying music: {e}")
298

```

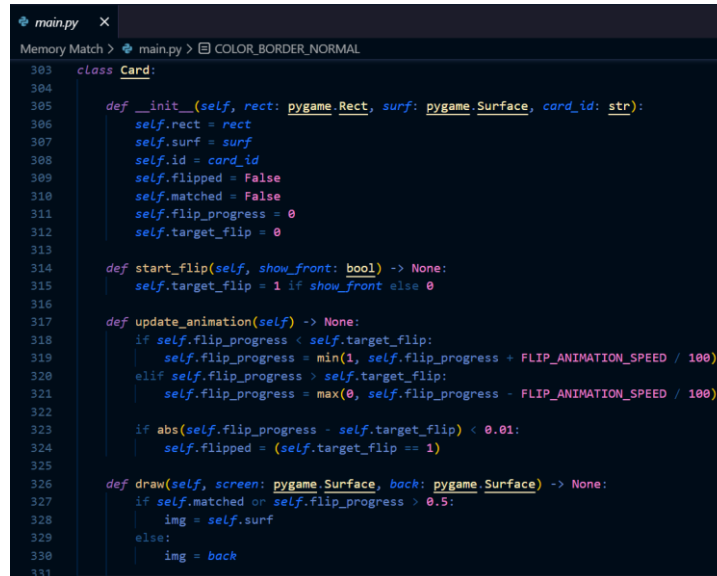
Explanation:

- **Attributes:** Stores the current volume levels for music and sound effects, and boolean flags to enable/disable them independently.
- **play_music():** Handles loading a music file and starting playback. It includes a check to see if the requested music is already playing, preventing unnecessary reloading. Volume is set using the stored `self.music_volume`.
- **play_sound():** Plays a sound effect using its dedicated volume level (`self.sfx_volume`), provided SFX are enabled.
- **set_music_volume():** Allows dynamic control of music volume, ensuring the value is clamped between `0.0` and `1.0` (Pygame's volume range).
- **toggle_music():** Mutes or unmutes the background music, utilizing `pygame.mixer.music.pause()` and `unpause()` for seamless toggling.

3.1.5 Card and Board Classes (Game Logic Core)

The Card and Board classes form the structural and interactive core of the memory game.

3.1.5.1 The Card Class



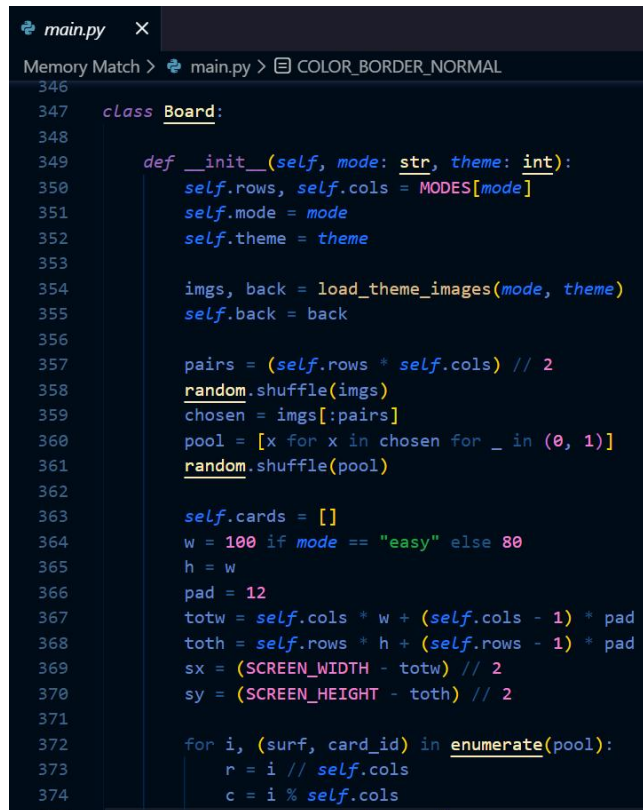
```
main.py x
Memory Match > main.py > COLOR_BORDER_NORMAL

303 class Card:
304
305     def __init__(self, rect: pygame.Rect, surf: pygame.Surface, card_id: str):
306         self.rect = rect
307         self.surf = surf
308         self.id = card_id
309         self.flipped = False
310         self.matched = False
311         self.flip_progress = 0
312         self.target_flip = 0
313
314     def start_flip(self, show_front: bool) -> None:
315         self.target_flip = 1 if show_front else 0
316
317     def update_animation(self) -> None:
318         if self.flip_progress < self.target_flip:
319             self.flip_progress = min(1, self.flip_progress + FLIP_ANIMATION_SPEED / 100)
320         elif self.flip_progress > self.target_flip:
321             self.flip_progress = max(0, self.flip_progress - FLIP_ANIMATION_SPEED / 100)
322
323         if abs(self.flip_progress - self.target_flip) < 0.01:
324             self.flipped = (self.target_flip == 1)
325
326     def draw(self, screen: pygame.Surface, back: pygame.Surface) -> None:
327         if self.matched or self.flip_progress > 0.5:
328             img = self.surf
329         else:
330             img = back
331
```

Explanation:

- `flip_progress`: A continuous float value used to track the animation state. It drives the visual representation of the card.
- `update_animation()`: The engine for the flip animation. It iteratively adjusts `flip_progress` until it reaches the `target_flip` state (0 or 1).
- `draw()`: The 3D Flip Illusion. This function implements a classic trick for card flips.
 1. It selects the image (`img`) to display based on whether the flip animation is past the halfway point (> 0.5).
 2. It calculates `scale_x = abs(1 - 2 * self.flip_progress)`, which makes the width of the card shrink to near zero at the midpoint, creating a *faux* 3D perspective effect.
 3. The image is scaled (`pygame.transform.smoothscale`) and drawn, centered horizontally, maintaining the height.

3.1.5.2 The Board Class



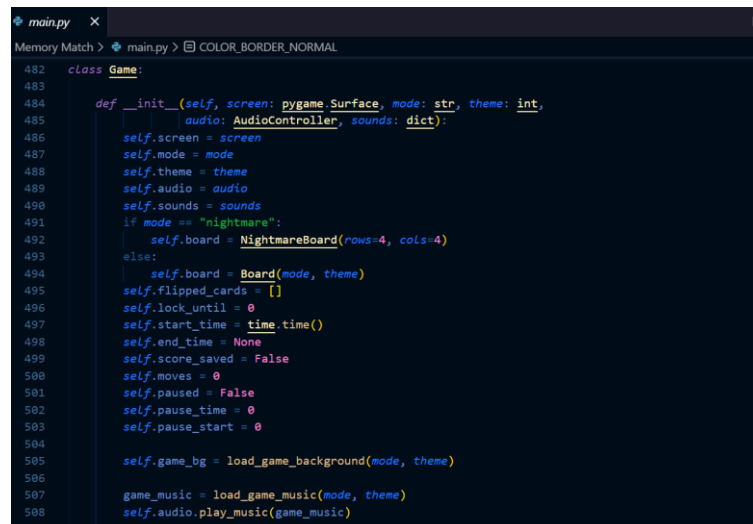
```
346
347 class Board:
348
349     def __init__(self, mode: str, theme: int):
350         self.rows, self.cols = MODES[mode]
351         self.mode = mode
352         self.theme = theme
353
354         imgs, back = load_theme_images(mode, theme)
355         self.back = back
356
357         pairs = (self.rows * self.cols) // 2
358         random.shuffle(imgs)
359         chosen = imgs[:pairs]
360         pool = [x for x in chosen for _ in (0, 1)]
361         random.shuffle(pool)
362
363         self.cards = []
364         w = 100 if mode == "easy" else 80
365         h = w
366         pad = 12
367         totw = self.cols * w + (self.cols - 1) * pad
368         toth = self.rows * h + (self.rows - 1) * pad
369         sx = (SCREEN_WIDTH - totw) // 2
370         sy = (SCREEN_HEIGHT - toth) // 2
371
372         for i, (surf, card_id) in enumerate(pool):
373             r = i // self.cols
374             c = i % self.cols
```

Explanation:

- Initialization: Loads the required assets. The core logic involves creating the card pool: selecting enough unique images for the mode's pairs, duplicating them, and then shuffling the final list to place them randomly across the grid.
- Layout Calculation: Calculates the necessary dimensions and spacing (w, h, pad) for the cards. It then determines the starting coordinates (sx, sy) required to visually center the entire grid of cards within the game window.
- Card Instantiation: Iterates through the shuffled pool, calculates the specific pixel coordinates for each card based on its grid position (r, c), and instantiates a Card object at that location.

3.1.6 Game Class (Game Manager)

The Game class orchestrates the gameplay, handling player input, the matching logic, time tracking, and rendering the game and post-game screens.

A screenshot of a code editor window titled 'main.py'. The editor shows a Python class named 'Game' with an '__init__' method. The method initializes various attributes: 'screen' (pygame.Surface), 'mode' (str), 'theme' (int), 'audio' (AudioController), and 'sounds' (dict). It then checks if the mode is 'nightmare', and if so, initializes 'board' as a 'NightmareBoard' with 4 rows and 4 columns. Otherwise, it initializes 'board' as a 'Board' with the given mode and theme. It also initializes 'flipped_cards' as an empty list, 'lock_until' as 0, 'start_time' as the current time, 'end_time' as None, 'score_saved' as False, 'moves' as 0, 'paused' as False, 'pause_time' as 0, and 'pause_start' as 0. Finally, it loads the game background, music, and plays the music.

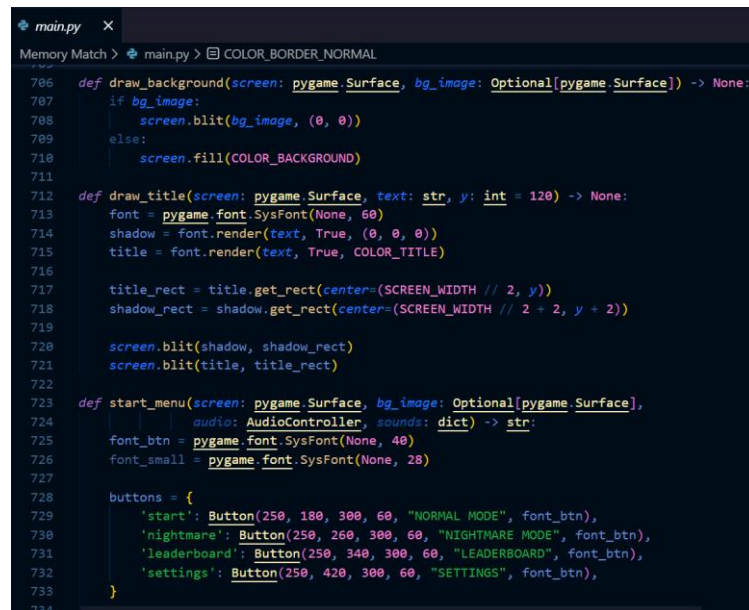
```
482 class Game:
483
484     def __init__(self, screen: pygame.Surface, mode: str, theme: int,
485                 audio: AudioController, sounds: dict):
486         self.screen = screen
487         self.mode = mode
488         self.theme = theme
489         self.audio = audio
490         self.sounds = sounds
491         if mode == "nightmare":
492             self.board = NightmareBoard(rows=4, cols=4)
493         else:
494             self.board = Board(mode, theme)
495         self.flipped_cards = []
496         self.lock_until = 0
497         self.start_time = time.time()
498         self.end_time = None
499         self.score_saved = False
500         self.moves = 0
501         self.paused = False
502         self.pause_time = 0
503         self.pause_start = 0
504
505         self.game_bg = load_game_background(mode, theme)
506
507         game_music = load_game_music(mode, theme)
508         self.audio.play_music(game_music)
```

Explanation:

- **Game State Initialization:** Sets up the game environment, including creating a Board (or NightmareBoard), initializing timers (start_time), and move counter.
- toggle_pause(): Controls the pause state. It records and accumulates the time spent paused in self.pause_time so that the final score time is accurate.
- click(): **Input Handling.** Checks if the clicked coordinate falls within an unflipped, unmatched card. If so, it updates the card's flip state, adds it to flipped_cards, and checks for a match if two cards are now revealed.
- check_match(): **Matching Logic.**
 - If the card IDs are equal, both cards are marked matched, and the game state is checked for overall completion (self.board.all_matched()).
 - If the IDs differ, self.lock_until is set. This prevents further clicks until the delay period is over, forcing the player to observe the failed attempt.
- update(): Called every frame. It updates card animations via self.board.update(). The critical logic here is handling the end of the lock period: once time.time() surpasses self.lock_until, the mismatched cards are automatically flipped back (card.start_flip(False)).
- calculate_stars(): Determines the star rating awarded to the player based on the time taken, referencing the thresholds defined in STAR_RULES.

3.1.7 Menu Functions (start_menu, difficulty_menu, etc.)

These functions manage the main user navigation through the game, utilizing the reusable Button and Slider classes (not fully detailed here) and utility drawing functions.



```

main.py x
Memory Match > main.py > COLOR_BORDER_NORMAL
706 def draw_background(screen: pygame.Surface, bg_image: Optional[pygame.Surface]) -> None:
707     if bg_image:
708         screen.blit(bg_image, (0, 0))
709     else:
710         screen.fill(COLOR_BACKGROUND)
711
712 def draw_title(screen: pygame.Surface, text: str, y: int = 120) -> None:
713     font = pygame.font.SysFont(None, 60)
714     shadow = font.render(text, True, (0, 0, 0))
715     title = font.render(text, True, COLOR_TITLE)
716
717     title_rect = title.get_rect(center=(SCREEN_WIDTH // 2, y))
718     shadow_rect = shadow.get_rect(center=(SCREEN_WIDTH // 2 + 2, y + 2))
719
720     screen.blit(shadow, shadow_rect)
721     screen.blit(title, title_rect)
722
723 def start_menu(screen: pygame.Surface, bg_image: Optional[pygame.Surface],
724               audio: AudioController, sounds: dict) -> str:
725     font_btn = pygame.font.SysFont(None, 40)
726     font_small = pygame.font.SysFont(None, 28)
727
728     buttons = {
729         'start': Button(250, 180, 300, 60, "NORMAL MODE", font_btn),
730         'nightmare': Button(250, 260, 300, 60, "NIGHTMARE MODE", font_btn),
731         'leaderboard': Button(250, 340, 300, 60, "LEADERBOARD", font_btn),
732         'settings': Button(250, 420, 300, 60, "SETTINGS", font_btn),
733     }
734

```

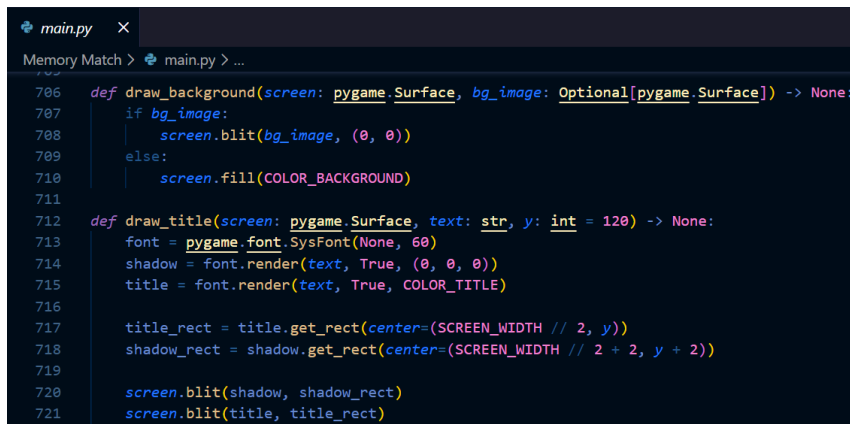
Explanation:

- **Menu Loop Structure:** Each menu function runs its own while True loop, handling user input events specific to its screen. It only exits when a navigation choice is made, returning a string or integer that the main application loop interprets as the next game state.
- **Button Interaction:** The menu logic relies on the Button class to simplify input processing. The button objects handle checking if the mouse is hovering over them (update_hover) and if a click occurred within their boundaries (is_clicked).
- **Navigation Flow:** The menus define the game's flow:
 - start_menu rightarrow chooses the high-level path.
 - Selecting **Normal Mode** rightarrow difficulty_menu.
 - Selecting a difficulty rightarrow theme_menu.
 - Selecting a theme rightarrow instantiation of the Game class.

3.2 Main Control Flow and Utility Classes

3.2.3 Drawing Utility Functions

These two fundamental functions are responsible for setting the background and consistently displaying the screen title across all menus.



```

706 def draw_background(screen: pygame.Surface, bg_image: Optional[pygame.Surface]) -> None:
707     if bg_image:
708         screen.blit(bg_image, (0, 0))
709     else:
710         screen.fill(COLOR_BACKGROUND)
711
712 def draw_title(screen: pygame.Surface, text: str, y: int = 120) -> None:
713     font = pygame.font.SysFont(None, 60)
714     shadow = font.render(text, True, (0, 0, 0))
715     title = font.render(text, True, COLOR_TITLE)
716
717     title_rect = title.get_rect(center=(SCREEN_WIDTH // 2, y))
718     shadow_rect = shadow.get_rect(center=(SCREEN_WIDTH // 2 + 2, y + 2))
719
720     screen.blit(shadow, shadow_rect)
721     screen.blit(title, title_rect)

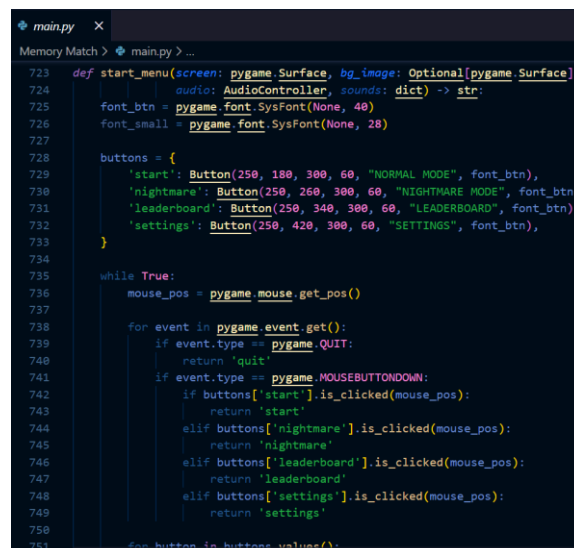
```

3.2.4 Menu Functions (User Interface Logic)

All menu functions adhere to a consistent **Menu Loop Pattern** to handle input, update button hover states, manage drawing, and finally return the next state/action upon button click.

A. start_menu (Main Menu)

This function serves as the primary entry point for user navigation.



```

723 def start_menu(screen: pygame.Surface, bg_image: Optional[pygame.Surface],
724               audio: AudioController, sounds: dict) -> str:
725     font_btn = pygame.font.SysFont(None, 40)
726     font_small = pygame.font.SysFont(None, 28)
727
728     buttons = {
729         'start': Button(250, 180, 300, 60, "NORMAL MODE", font_btn),
730         'nightmare': Button(250, 260, 300, 60, "NIGHTMARE MODE", font_btn),
731         'leaderboard': Button(250, 340, 300, 60, "LEADERBOARD", font_btn),
732         'settings': Button(250, 420, 300, 60, "SETTINGS", font_btn),
733     }
734
735     while True:
736         mouse_pos = pygame.mouse.get_pos()
737
738         for event in pygame.event.get():
739             if event.type == pygame.QUIT:
740                 return 'quit'
741             if event.type == pygame.MOUSEBUTTONDOWN:
742                 if buttons['start'].is_clicked(mouse_pos):
743                     return 'start'
744                 elif buttons['nightmare'].is_clicked(mouse_pos):
745                     return 'nightmare'
746                 elif buttons['leaderboard'].is_clicked(mouse_pos):
747                     return 'leaderboard'
748                 elif buttons['settings'].is_clicked(mouse_pos):
749                     return 'settings'
750
751         for button in buttons.values():

```

- **Design Pattern:** It operates within an infinite loop (while True) until an action string is explicitly **returned** ('start', 'nightmare', 'leaderboard', 'settings', or 'quit').
- **Input Handling:** The MOUSEBUTTONDOWN event check delegates to the Button.is_clicked(mouse_pos) method. This abstraction simplifies the menu logic significantly.

B. Difficulty_Menu And Theme_Menu

These menus follow the same loop pattern but are designed to select specific game parameters.

C. leaderboard_screen and nightmare_leaderboard (Score Display)

These functions retrieve, filter, and display persistent score data, focusing heavily on structured drawing.

```
main.py x
Memory Match > main.py > ...
973 def leaderboard_screen(screen: pygame.Surface, bg_image: Optional[pygame.Surface],
974                          audio: AudioController, sounds: dict) -> None:
975     font_title = pygame.font.SysFont(None, 50)
976     font_tab = pygame.font.SysFont(None, 32)
977     font_entry = pygame.font.SysFont(None, 24)
978     font_small = pygame.font.SysFont(None, 20)
979
980     back_btn = Button(30, 30, 100, 40, "Back", font_small)
981
982     mode_buttons = {}
983     mode_texts = ["All", "Easy", "Medium", "Hard"]
984     for i, text in enumerate(mode_texts):
985         btn = Button(200 + i * 110, 110, 100, 35, text, font_small)
986         mode_buttons[text.lower()] = btn
987
988     selected_mode = "all"
989
990     while True:
991         mouse_pos = pygame.mouse.get_pos()
992
993         for event in pygame.event.get():
994             if event.type == pygame.QUIT:
995                 return
996             if event.type == pygame.MOUSEBUTTONDOWN:
997                 if back_btn.is_clicked(mouse_pos):
998                     return
999                 for mode_name, btn in mode_buttons.items():
1000                     if btn.is_clicked(mouse_pos):
1001                         selected_mode = mode_name
```

- **Dynamic Filtering:** The `selected_mode` variable is used to dynamically filter the scores retrieved via `get_top_scores()`, which accesses the CSV data.
- **Structured Drawing:** The scores are rendered using fixed horizontal positions (`header_x`) and dynamically incrementing vertical positions (`entry_y`) to create the appearance of a neat, aligned table.

D. Settings_screen (Settings Menu)

This screen manages volume controls using the custom Slider class and audio state using Toggle Buttons.

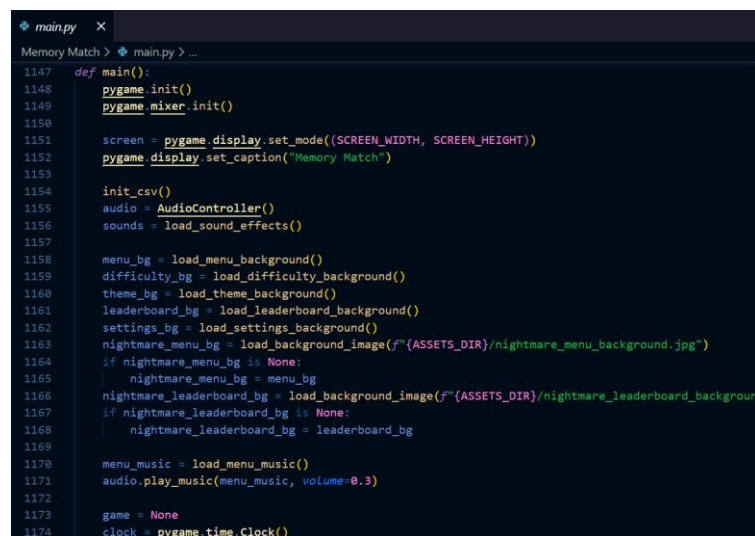
```
main.py x
Memory Match > main.py > ...
1089 def settings_screen(screen: pygame.Surface, bg_image: Optional[pygame.Surface],
1090                      audio: AudioController, sounds: dict) -> None:
1091     """Settings screen with volume controls."""
1092     font_title = pygame.font.SysFont(None, 50)
1093     font_label = pygame.font.SysFont(None, 28)
1094     font_small = pygame.font.SysFont(None, 24)
1095
1096     back_btn = Button(30, 30, 100, 40, "Back", font_small)
1097
1098     music_slider = Slider(200, 230, 350, 0, 1, audio.music_volume, "Music Volume")
1099     sfx_slider = Slider(200, 300, 350, 0, 1, audio.sfx_volume, "Sound Effects Volume")
1100
1101     music_toggle = Button(200, 360, 200, 45,
1102                          "Music: ON" if audio.music_enabled else "Music: OFF", font_label)
1103     sfx_toggle = Button(420, 360, 200, 45,
1104                       "SFX: ON" if audio.sfx_enabled else "SFX: OFF", font_label)
1105
1106     while True:
1107         mouse_pos = pygame.mouse.get_pos()
1108
1109         for event in pygame.event.get():
1110             if event.type == pygame.QUIT:
1111                 return
1112             if event.type == pygame.MOUSEBUTTONDOWN:
1113                 if back_btn.is_clicked(mouse_pos):
1114                     return
1115                 if music_toggle.is_clicked(mouse_pos):
1116                     audio.toggle_music()
1117                 if sfx_toggle.is_clicked(mouse_pos):
1118                     audio.toggle_sfx()
```

- **Slider Interaction:** The Slider class encapsulates its own event processing (`handle_event`). When a slider's value changes, the code immediately calls the corresponding method in the global AudioController to apply the new volume setting.

- **Toggle Interaction:** The music/SFX toggles directly call the AudioController methods to change the state and simultaneously update their displayed text to reflect the current ON/OFF status.

3.2.5 The Main Function (Application Control Flow)

The main() function is the Main Loop that initializes the Pygame environment and controls the game's overall flow by managing state transitions based on the return values of the menu functions.



```

1147 def main():
1148     pygame.init()
1149     pygame.mixer.init()
1150
1151     screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
1152     pygame.display.set_caption("Memory Match")
1153
1154     init_csv()
1155     audio = AudioController()
1156     sounds = load_sound_effects()
1157
1158     menu_bg = load_menu_background()
1159     difficulty_bg = load_difficulty_background()
1160     theme_bg = load_theme_background()
1161     leaderboard_bg = load_leaderboard_background()
1162     settings_bg = load_settings_background()
1163     nightmare_menu_bg = load_background_image(f"{ASSETS_DIR}/nightmare_menu_background.jpg")
1164     if nightmare_menu_bg is None:
1165         nightmare_menu_bg = menu_bg
1166     nightmare_leaderboard_bg = load_background_image(f"{ASSETS_DIR}/nightmare_leaderboard_background.jpg")
1167     if nightmare_leaderboard_bg is None:
1168         nightmare_leaderboard_bg = leaderboard_bg
1169
1170     menu_music = load_menu_music()
1171     audio.play_music(menu_music, volume=0.3)
1172
1173     game = None
1174     clock = pygame.time.Clock()

```

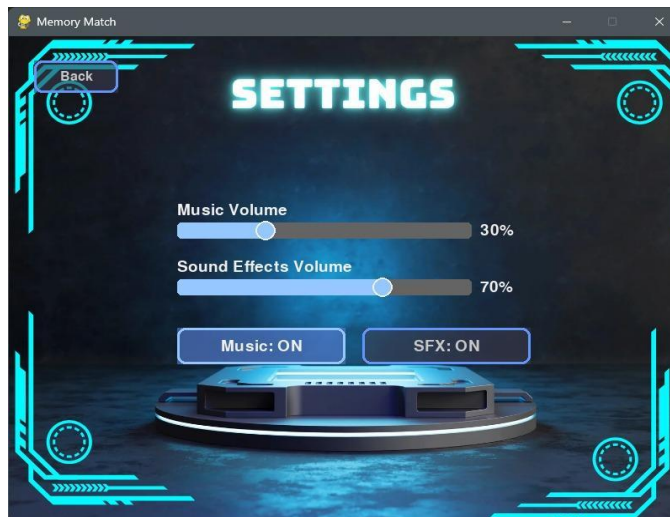
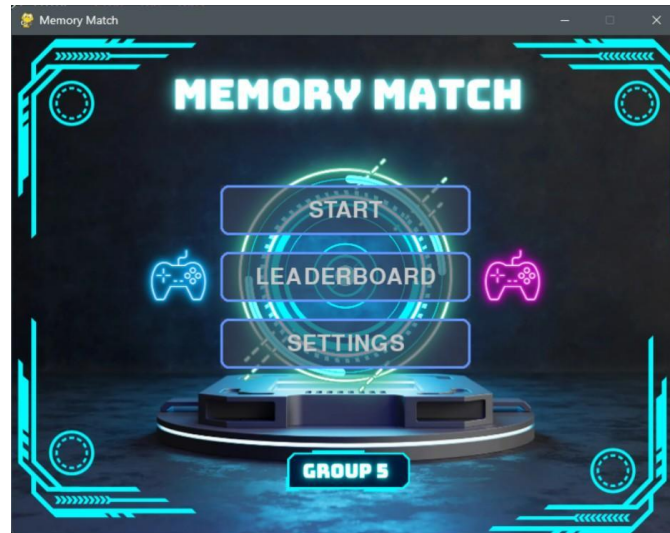
Key Control Flow Mechanisms:

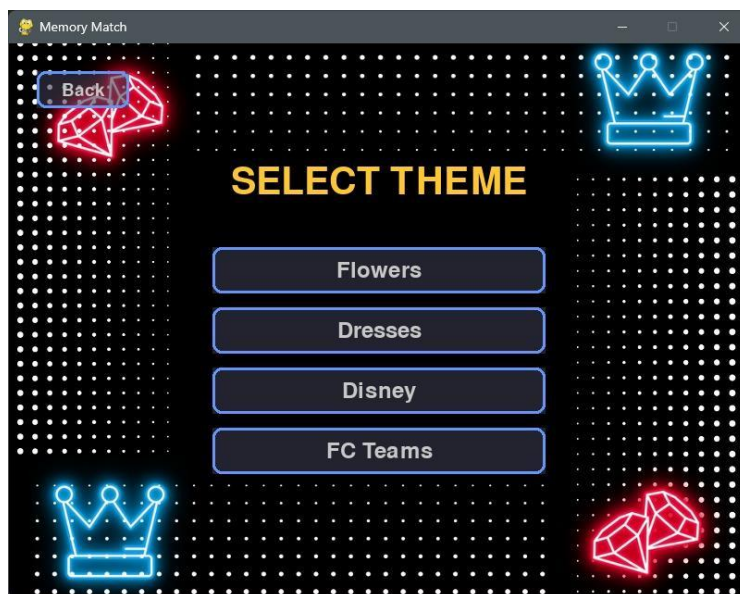
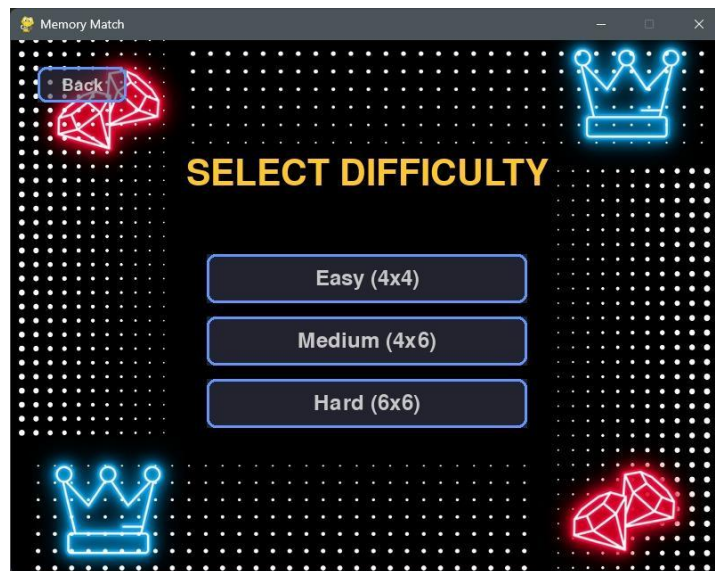
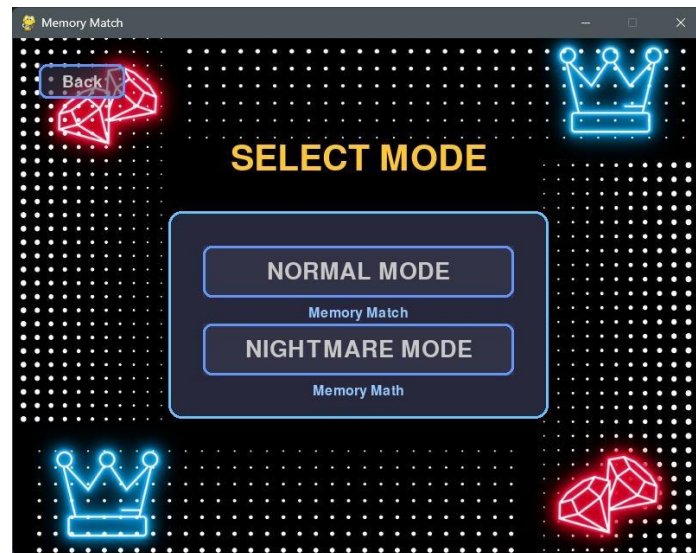
1. **Outer Loop (while True):** This loop manages the highest level of navigation, primarily returning to the start_menu after visiting a static screen (Leaderboard, Settings).
2. **Nested Loops (Baris 568, 614):** Starting a game (Normal or Nightmare) requires entering a nested while True loop to handle the sequence of menu choices (e.g., Difficulty $\rightarrow$ Theme).
3. **The Game Loop (Baris 584, 632):** The innermost while running loop is where the actual gameplay happens.
 - It contains the event handlers (mouse clicks, ESC key).
 - It calls game.update() for logic processing.
 - It calls game.draw() for rendering.
 - It only exits (running = False) when the game is completed or the user chooses to return to the menu from the pause screen.
4. **State Transition via return and break:**
 - Menu functions use **return 'action'** to pass control to the main() function.

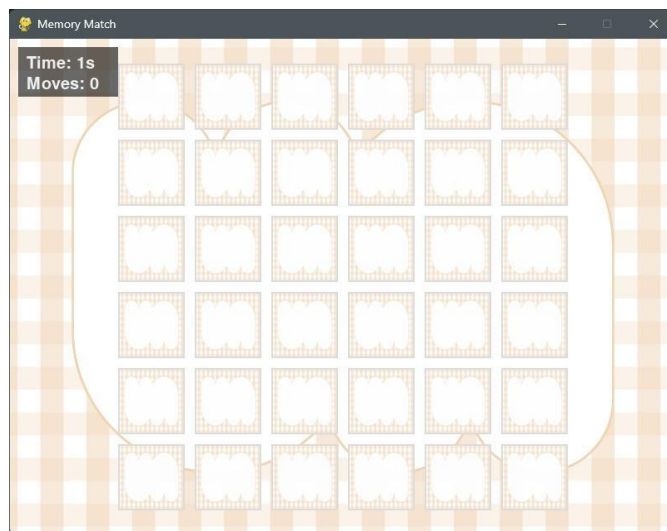
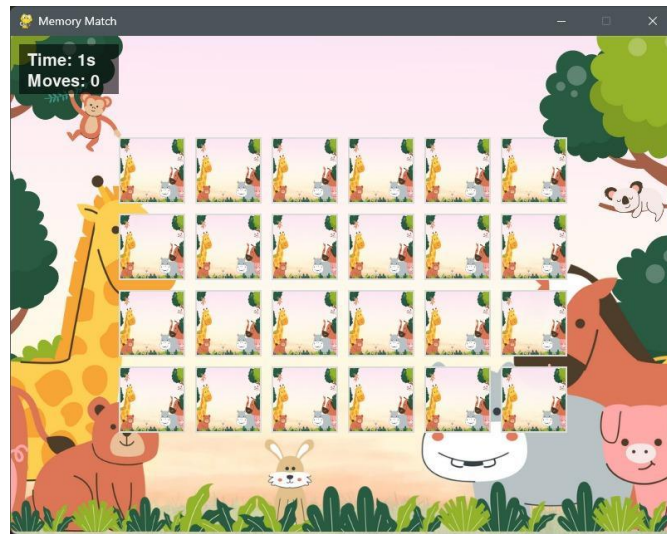
- The main() function uses **continue** to return to the start_menu (for static screens like settings) or **break** to exit the inner selection loops and return to a higher level.

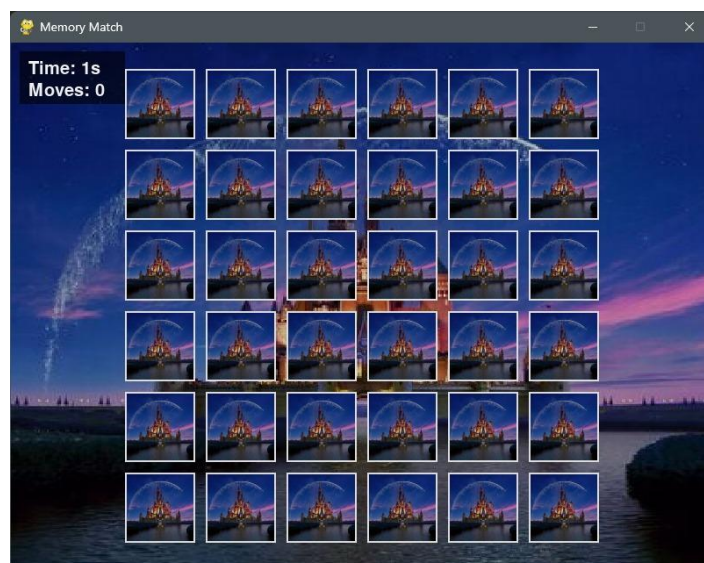
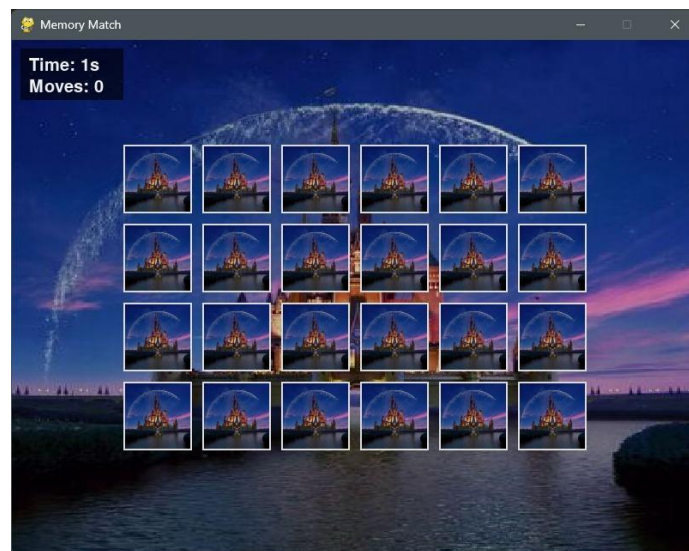
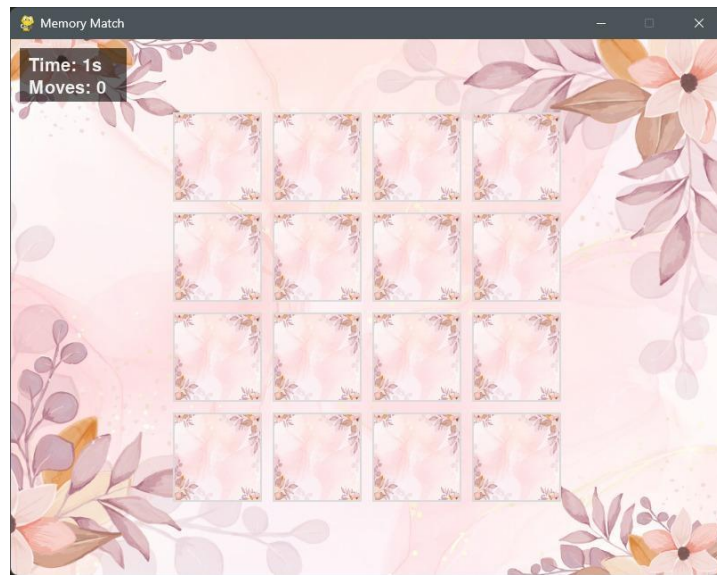
This structure effectively implements a **State Machine**, ensuring that the program is always in a clear, well-defined state (e.g., 'In Main Menu', 'Selecting Difficulty', or 'Playing Game').

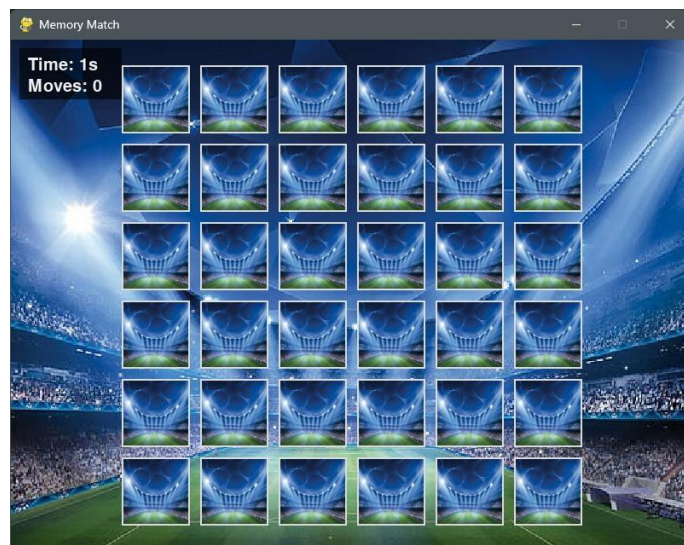
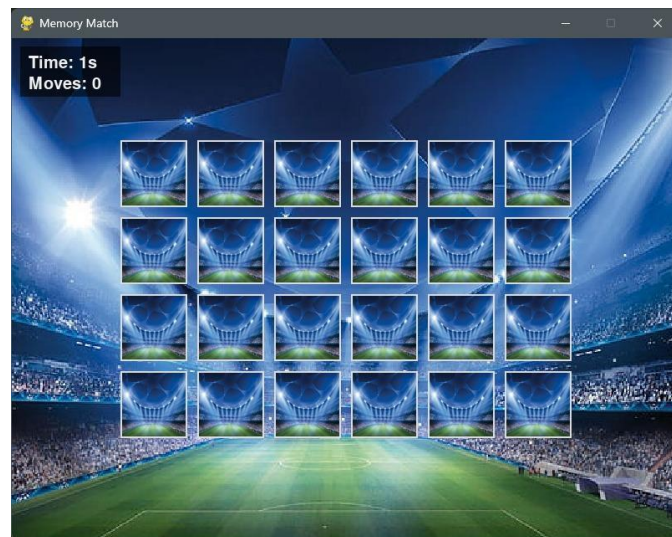
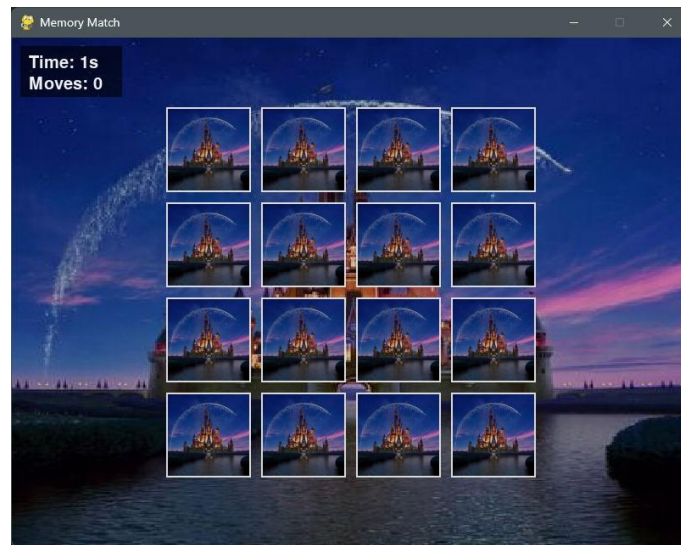
3. 2 APPLICATION SCREENSHOTS

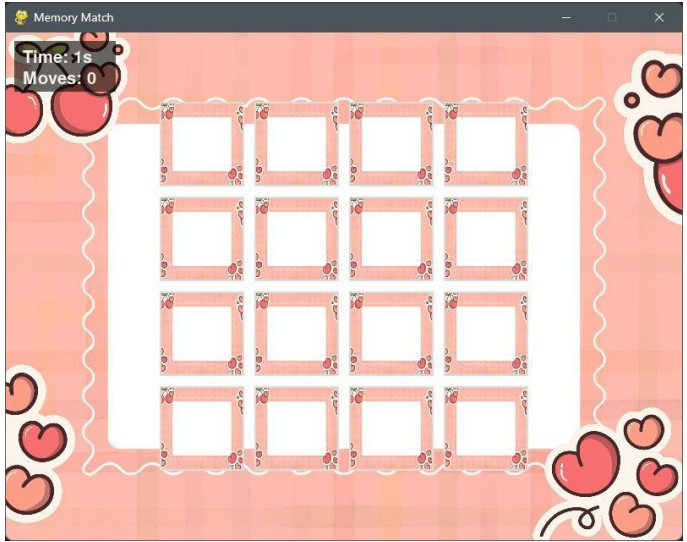
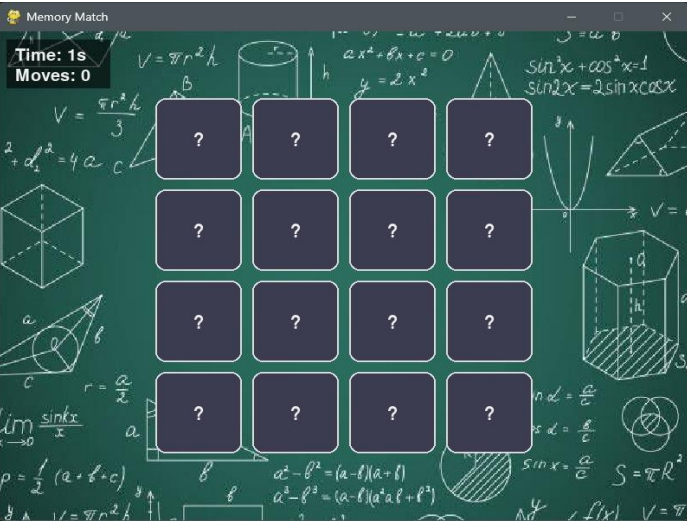
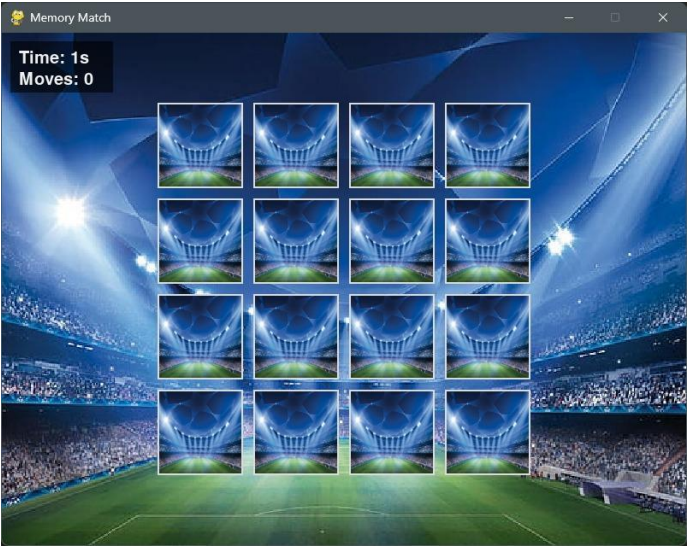












CHAPTER 4

ATTACHMENT

4.1Embed File

Embed below is the complete folder of assets and the full code of Memory Math.

<https://drive.google.com/file/d/1SNhAaLw0f2UTGeeyBPOOPPQW05R7l0kT/view?usp=sharing>



REFERENCES

- Baddeley, A. (2012). *Working Memory: Theories, Models, and Controversies*. Annual Review of Psychology, 63(1), 1–29.
- Boot, W. R., Kramer, A. F., Simons, D. J., Fabiani, M., & Gratton, G. (2008). *The Effects of Video Game Playing on Attention, Memory, and Executive Control*. Acta Psychologica, 129(3), 387–398.
- Buckley, P., & Doyle, E. (2016). *Gamification and Student Motivation*. Interactive Learning Environments, 24(6), 1162–1175.
- Granic, I., Lobel, A., & Engels, R. C. (2014). *The Benefits of Playing Video Games*. American Psychologist, 69(1), 66–78.
- Kurniawan, D., & Wibowo, A. (2020). *Pengembangan Game Edukasi Berbasis Pygame untuk Pembelajaran Interaktif*. Jurnal Teknologi Informasi, 14(2), 55–62.
- Lutz, M., & Ascher, D. (2013). *Learning Python* (5th ed.). O'Reilly Media.
- Pygame Community. (2023). *Pygame Documentation*. Retrieved from <https://www.pygame.org/docs/>
- Squire, K. (2011). *Video Games and Learning: Teaching and Participatory Culture in the Digital Age*. MIT Press.
- Wang, A. I. (2015). *Gamification in Teaching and Learning: Practices and Perspectives*. Springer.
- Widodo, S. A., & Utami, E. (2019). *Perancangan Game Edukasi Menggunakan Pendekatan OOP*. Jurnal Ilmiah Teknologi dan Informasi, 8(1), 30–38.